Source code was made in this repository: https://github.com/Ivan-projects-dev/NLP-HW2

The whole code is divided in 3 files (main.py, data_analysis.py, model.py), each is runned separately but consequently. Python 3.7 was used.

Dataset: acquisition, preprocessing, analysis (max 6 points)

1. Download the **GoEmotions** dataset from: https://github.com/google-research/google-research/tree/master/goemotions `./data/{train,dev,test}.tsv` (ignore the 3rd column in the TSV files)
2. **Convert** the original train/dev/test files that use **27** fine-grained emotions (+neutral) **into** train/dev/test files that use only **6** classes of basic emotions (+neutral) – see `./data/ekman_mapping.json`
3. Note that about **15%** of samples in the dataset are labelled with more than one emotion – remove such samples from the resulting train/dev/test files
4. Do basic text **tokenization** of your choice (e.g. regex, NLTK, spaCy, Stanza)
5. Conduct shallow **data analysis** (e.g. samples per class; unigram/bigram TOP frequencies per class; potential impact of frequency-based vocabulary pruning – vocabulary size)
6. Consider basic text **clean-up** or **normalization** (e.g. numbers=>placeholders, wordforms=>lemmas)

This part was fully done in **main.py**. To test everything properly, a new file with edited data is created during each step. At start, samples with more that one emotion were removed, and only after that conversion to Ekman mapping was done.

```
PS C:\Users\User\Desktop\NLP-HW2> py -3.7 main.py
train.tsv -> train_clean.tsv | 36308 rows
dev.tsv -> dev_clean.tsv | 4548 rows
test.tsv -> test_clean.tsv | 4590 rows
train_clean.tsv -> train_ekman.tsv | 36308 rows
dev_clean.tsv -> dev_ekman.tsv | 4548 rows
test_clean.tsv -> test_ekman.tsv | 4590 rows
train_ekman.tsv -> train_tokenized.tsv
dev_ekman.tsv -> dev_tokenized.tsv
test_ekman.tsv -> test_tokenized.tsv
```

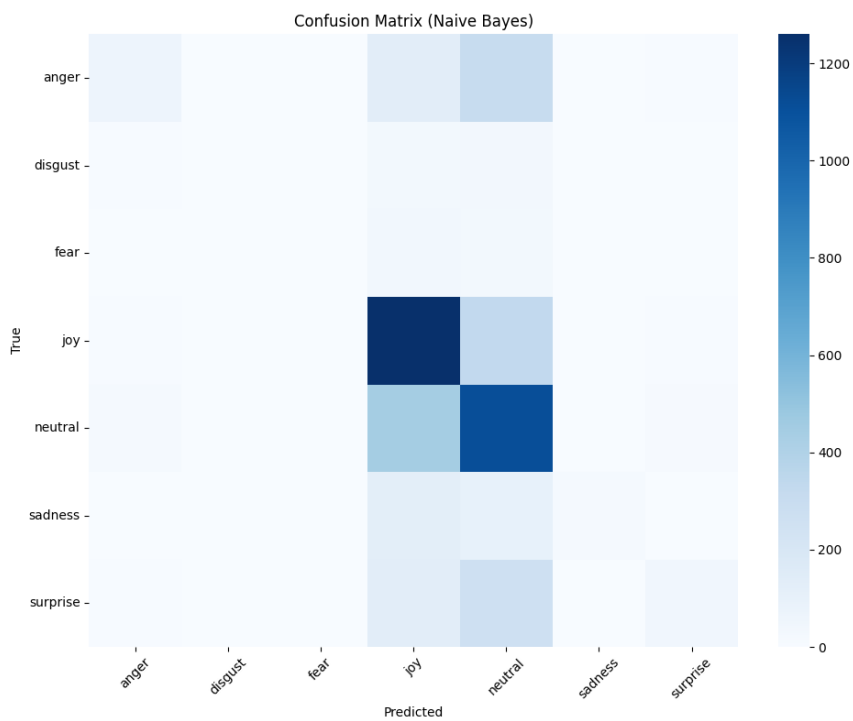
Training: a one-of multinomial Naïve Bayes classifier (max 7 points, i.e., +1)

- Train a 7-class classifier = 6 basic emotions + neutral
- Using your modified **`train.tsv`**

This part was implemented in **model.py**.

```
PS C:\Users\User\Desktop\NLP-HW2> py -3.7 model.py

===== Naive Bayes =====

DEV RESULTS:
Accuracy: 0.5551890941072999
Macro F1: 0.2595923938145102
              precision    recall  f1-score   support

       anger       0.43      0.09      0.16       485
     disgust       0.50      0.02      0.03        61
        fear       0.00      0.00      0.00        66
         joy       0.61      0.79      0.69      1668
     neutral       0.50      0.68      0.58      1592
     sadness       0.76      0.10      0.18       241
    surprise       0.62      0.11      0.18       435

    accuracy                           0.56      4548
   macro avg       0.49      0.26      0.26      4548
weighted avg       0.55      0.56      0.50      4548
```

```
TEST RESULTS:
Accuracy: 0.5474945533769063
Macro F1: 0.26655800962487947
              precision    recall  f1-score   support

       anger       0.59      0.13      0.21       520
     disgust       1.00      0.04      0.08        76
        fear       0.00      0.00      0.00        77
         joy       0.58      0.79      0.67      1603
     neutral       0.51      0.69      0.59      1606
     sadness       0.81      0.08      0.15       259
    surprise       0.56      0.10      0.17       449

    accuracy                           0.55      4590
   macro avg       0.58      0.26      0.27      4590
weighted avg       0.56      0.55      0.49      4590


Confusion matrix saved to file
```



Classifier worked with 6 basic emotions + neutral. We can see significant difference between accuracy and macro F1-score, which indicates a class imbalance and inconsistent classification quality. There are also many cross-errors between **joy** and

neutral, and these emotions are more often predicted. Emotions of **fear** and **disgust** have extremely low recall, so is their F1-score. Despite comparatively good accuracy of 55% the model shows low quality for separate classes, which can be explained by low F1-score.

Create and evaluate a text classifier

Evaluation: for each class, and overall (max 9 points, i.e., +1)

- Using your modified `test.tsv`
- Calculate and analyse a confusion matrix, precision/recall, F_1-score, average accuracy (micro vs. macro)
  - Compare your results to Table 6 in https://arxiv.org/pdf/2005.00547
- Analyse your results, **e.g.**:
  - What is the impact of vocabulary pruning?
  - Does text clean-up/normalization or improved tokenization improve results?
  - Other observations, conclusions?

Extra task of your choice: for a bonus point (max 10 points)

- EITHER combine uni-grams and bi-grams in your vocabulary (feature vector) – e.g. `CountVectorizer`
  - AND compare to TF-IDF vectorization – e.g. TfidfVectorizer
- OR train and use a BPE tokenizer (see e.g. TextPreprocessing.ipynb)
- OR use another machine-learning approach (you might also need to use `dev.tsv`)
- Compare these results to your best previous result

This part was fully completed in **data_analysis.py**. The first additional task was chosen**.**

Vocabulary size: 23812

```
PS C:\Users\User\Desktop\NLP-HW2> py -3.7 data_analysis.py
Class distribution:
joy         12920
neutral     12823
anger        3878
surprise     3553
sadness      2121
fear          515
disgust       498
Name: label, dtype: int64

Top words per class:

anger: [('the', 1583), ('i', 1541), ('to', 1185), ('you', 1110), ('it', 1011), ('that', 940), ('is', 813), ('and', 794), ('of', 669), ('not', 595)]

disgust: [('the', 224), ('i', 193), ('is', 165), ('it', 147), ('that', 124), ('and', 110), ('to', 101), ('this', 87), ('you', 83), ('disgusting', 77)]

fear: [('i', 335), ('the', 196), ('to', 164), ('that', 151), ('it', 136), ('is', 120), ('and', 99), ('of', 99), ('this', 97), ('me', 88)]

joy: [('i', 7095), ('the', 4900), ('you', 3558), ('to', 3462), ('it', 3168), ('that', 2660), ('and', 2529), ('is', 2253), ('name', 2140), ('this', 2071)]

neutral: [('the', 5586), ('i', 3950), ('to', 3812), ('name', 2975), ('you', 2821), ('and', 2760), ('it', 2734), ('that', 2600), ('is', 2428), ('of', 2134)]

sadness: [('i', 1368), ('the', 817), ('to', 710), ('it', 577), ('that', 558), ('and', 462), ('you', 422), ('is', 407), ('of', 392), ('this', 380)]

surprise: [('i', 1961), ('the', 1610), ('to', 1019), ('you', 1015), ('that', 964), ('it', 869), ('is', 810), ('name', 607), ('and', 605), ('this', 587)]
```

Copied from console (output continuation):

Top bigrams per class:

- anger: [(('i', 'don'), 174), (('you', 're'), 150), (('in', 'the'), 121), (('this', 'is'), 101), (('of', 'the'), 95), (('i', 'hate'), 90), (('to', 'be'), 90), (('if', 'you'), 67), (('but', 'i'), 65), (('you', 'can'), 64)]
- disgust: [(('the', 'worst'), 34), (('this', 'is'), 30), (('in', 'the'), 22), (('i', 've'), 18), (('to', 'be'), 16), (('is', 'the'), 15), (('it', 'was'), 15), (('on', 'the'), 14), (('you', 're'), 13), (('is', 'disgusting'), 13)]
- fear: [(('i', 'was'), 33), (('this', 'is'), 20), (('i', 'afraid'), 18), (('i', 'scared'), 16), (('to', 'be'), 16), (('scared', 'to'), 14), (('i', 'am'), 14), (('afraid', 'of'), 13), (('i', 'have'), 13), (('but', 'i'), 12))
- joy: [(('thank', 'you'), 695), (('i', 'love'), 670), (('this', 'is'), 453), (('for', 'the'), 411), (('you', 're'), 380), (('thanks', 'for'), 349), (('in', 'the'), 338), (('of', 'the'), 317), (('but', 'i'), 269), (('i', 'like'), 255)]
- neutral: [(('in', 'the'), 472), (('of', 'the'), 340), (('to', 'be'), 304), (('you', 're'), 297), (('NUM', 'NUM'), 295), (('on', 'the'), 240), (('this', 'is'), 237), (('name', 'is'), 235), (('if', 'you'), 219), (('i', 'think'), 205)]
- sadness: [(('i', 'sorry'), 87), (('to', 'be'), 73), (('i', 'was'), 71), (('of', 'the'), 68), (('in', 'the'), 65), (('i', 'feel'), 58), (('this', 'is'), 58), (('i', 'am'), 51), (('it', 'was'), 51), (('sorry', 'i'), 48)]
- surprise: [(('in', 'the'), 134), (('i', 'don'), 132), (('do', 'you'), 121), (('i', 'was'), 108), (('are', 'you'), 99), (('of', 'the'), 91), (('i', 've'), 84), (('you', 're'), 78), (('don', 'know'), 75), (('it', 'was'), 75)]

The dataset is dominated by **joy** and **neutral** classes, which negatively affects classification as was shown in previous stage of the project. Most frequent words (i, NUM, this, you etc.) do not carry strong semantic meaning and can be used (and are used) regardless of emotions. These words add noise and therefore reduce model quality. Most of these words cannot be removed without loss of context.

However, some emotion-specific words still appear:

- disgust: worst, disgusting
- fear: scared, afraid
- joy: thanks, thank, love
- sadness: sorry